Milestone 3. routes.py Development

In this milestone, the primary focus was on setting up all major FastAPI routes for the app, handling user inputs efficiently, integrating AI generation workflows, and ensuring smooth interaction between backend processing and frontend templates.
This step-by-step approach created a stable foundation for processing prompts, generating comics, and providing a seamless user experience.

Activity 3.1: Writing the Main Application Logic in routes.py

Defining Core routes in routes.py:

Separate routes were established for each of ComicCraft's core functionalities:

- / → Loads the **homepage** (index.html) where users can submit their story details.
- /generate → Handles **form submission**, processes the input using AI, generates the comic panels, and returns the **comic preview** page.
- /generate-comic/json → An API route that accepts **JSON payloads**, triggers comic generation, and returns **comic layout data** and the generated PDF path.
- /export-success → Displays a **success confirmation page** after the comic is downloaded.
- /test-image → A developer utility route to **test image generation** from a direct prompt without full comic creation.

Each route serves as a distinct entry point for different types of user interaction — through browser forms or direct API requests.

Set up Route Handlers for Each Feature

Each route handler captures and processes user inputs using FastAPI's form and request body mechanisms:

- For /generate, user inputs are captured using Form(...) parameters for:
  - Story prompt
  - Character name
  - Setting
  - Story tone
  - Art style
- For /generate-comic/json, inputs are captured from a JSON body using a PromptRequest model with Pydantic.

Form data and JSON payloads are correctly extracted and passed into backend functions (generate_outline, generate_story, generate_image, build_comic_layout, save_pdf) ensuring smooth

AI processing based on user details.

The route handlers manage error handling via FastAPI's HTTPException system and use try-except blocks to gracefully handle unexpected failures like API errors or JSON decode errors.

Integrate AI workflows in each Route

Within each relevant route, the core AI functionalities were integrated:

In /generate and /generate-comic/json:

- generate_outline() is called to create a 5-panel structured story.
- generate_story() is called to write narration and dialogues.
- generate_image() is called iteratively to generate images for each panel.
- build_comic_layout() structures the final panel layout.
- save_pdf() compiles the story and illustrations into an exportable PDF.

Each AI-generated output is processed and dynamically passed to HTML templates (comic_preview.html) or returned as a JSON response for API clients.

This ensures that users receive a complete, coherent comic experience, while developers and testers can interact with the backend directly through structured API responses.

Comic Generation Route:

```python

@router.post("/generate", response_class=HTMLResponse)
async def generate_comic(
    request: Request,
    prompt: str = Form(...),
    character_name: str = Form(...),
    setting: str = Form(...),
    tone: str = Form(...),
    style: str = Form(...)
):
    try:
        # Combine user input into a single full prompt
        full_prompt = (
            f"{prompt}\n"
            f"The main character is {character_name}. "
            f"The setting is a {setting}. "
            f"The tone is {tone}. The art style is {style}."
        )

        # Step 1: Generate panel outline
        outline = generate_outline(full_prompt)

        if not isinstance(outline, list) or not all("image_prompt" in panel for panel in outline):
            raise ValueError("Invalid outline structure from Gemini response.")

        # Step 2: Generate story
        full_story = generate_story(outline)

        # Step 3: Generate images
        images = [generate_image(panel["image_prompt"]) for panel in outline]

        # Step 4: Build layout
        layout = build_comic_layout(images, full_story, outline)

        # Step 5: Export to PDF
        pdf_path = save_pdf(layout)
        web_pdf_path = "/" + pdf_path.replace("\\", "/")  # Make path safe for web

        return templates.TemplateResponse("comic_preview.html", {
            "request": request,
            "layout": layout,
            "pdf_path": web_pdf_path
        })

    except Exception as e:
        traceback.print_exc()
        raise HTTPException(status_code=500, detail=str(e))

```

Export Success Route:

```
70
71 @router.get("/export-success", response_class=HTMLResponse)
72 async def export_success(request: Request, pdf_path: str):
73     return templates.TemplateResponse("export_success.html", {
74         "request": request,
75         "pdf_path": pdf_path
76     })
77
78
```

Image generation Route:

```
120
121 @router.get("/test-image")
122 async def test_image(prompt: str = "A futuristic city at sunset, sci-fi, cinematic, artstation"):
123     try:
124         image_path = generate_image(prompt)
125         return {"message": "Image generated successfully", "path": image_path}
126     except Exception as e:
127         raise HTTPException(status_code=500, detail=str(e))
128
```